Task 3 – Mathematical complexity analysis

Linear search

Let n be the number of records. If the target is at zero-based position k, comparisons C(n)=k+1. If absent, C(n)=n. Thus best C(n)=1; worst C(n)=n. If a successful target is equally likely to occupy any position, average comparisons are (1+2+…+n)/n = [n(n+1)/2]/n = (n+1)/2. Therefore search time is Θ(n) on average and worst case, with best-case Θ(1). No preprocessing is needed, so total cost per search remains Θ(n).

Recursive binary search

For n>1, one middle comparison is made and only one subproblem of size approximately n/2 is searched. The recurrence is T(n)=T(⌊n/2⌋)+c, with T(0)=c0. After j steps, the interval is approximately n/2^j. Stop when n/2^j ≤ 1, giving j ≥ $\log_2$ n. Hence T(n)=Θ(log n). Best case is Θ(1) when the first midpoint matches; average and worst cases are Θ(log n). Sorting costs Θ(n log n) using a comparison sort. If the data are initially unsorted, one-time total cost for q searches is Θ(n log n + q log n); if already maintained in sorted order, a query costs Θ(log n), but inserts may require maintenance.

Hash-table search

Building the table takes Θ(n) expected time and Θ(n) additional space. Under a well-distributed hash function and controlled load factor, expected query time is Θ(1); the worst case is Θ(n) if many keys collide. For q queries, expected total cost after building is Θ(n+q), versus Θ(qn) for repeated linear search. In this implementation, a Python dictionary is used. The benchmark counts a dictionary lookup as one abstract lookup operation; its internal collision probes are not exposed by Python and therefore are not directly counted.